Software Requirements Specification (SRS)

System: Jima Aba Jifar — Cafe & Restaurant Management System
Document: SRS v1.1 · **Updated:** August 19, 2026
Standard: Structured after IEEE 830

1. Introduction

1.1 Purpose

This SRS specifies the complete requirements — functional and non-functional — for the Jima Aba Jifar Restaurant Management System. It is intended for developers, testers, and stakeholders taking over or extending the system.

1.2 Scope

The system digitizes end-to-end restaurant operations: table-side ordering (POS), kitchen workflow, cashiering, inventory and procurement, staff and branch administration, and management analytics. It is a web application accessed from browsers on desktops and tablets.

1.3 Definitions

Term	Meaning
POS	Point of Sale — the screen waiters use to compose orders.
Order Board	Live kanban-style view of order progress.
JWT	JSON Web Token used for stateless authentication.

Item Request	An internal request for stock items from the store.

2. Overall Description

2.1 Product Perspective

Three-tier logical architecture: a **Next.js 15 / React 19 frontend**, a **NestJS REST API backend**, and a **PostgreSQL database** accessed through TypeORM. The registered Abajiraf web artifact runs the actual source in `artifacts/nextjs-app`. The frontend calls the backend through `/backend/api/*`; Next.js rewrites requests to the backend's `/api/*`. The separate `artifacts/api-server` is not the restaurant API.

2.2 User Classes

Eight roles: admin, owner, manager, coordinator, waiter, chef, cashier, storekeeper (see FRS §2). Users are non-technical restaurant staff; the UI must remain simple, large-touch-friendly, and in plain language.

2.3 Operating Environment

Layer	Technology
Frontend	Next.js 15 (App Router), React 19, TypeScript, Tailwind CSS, Zustand (auth store), Recharts, Axios
Backend	NestJS 10, TypeScript, Passport-JWT, bcrypt, TypeORM
Database	PostgreSQL 14+ (TypeORM `synchronize: true` in development)
Runtime	Node.js 18+, pnpm monorepo workspaces

2.4 Constraints

- Protected API access requires JWT. Login and health are public; seed operations are administrative.
- The current backend still enables TypeORM schema synchronization. Production is blocked until synchronization is disabled there and a tested migration/rollback baseline exists.

- Realtime behavior is implemented by 10-second polling, not WebSockets.
- Currency is Ethiopian Birr (ETB); demo data uses Ethiopian dishes and names.

3. Functional Requirements

The complete functional catalogue is maintained in the FRS (document 01). Summary of subsystems:

Subsystem	Capabilities
Auth	Login, JWT issuance, role-based menus and guards
Orders / POS	Create, append items, confirm + assign chef, status lifecycle, filters, auto-refresh
Kitchen	Chef queues, accept/preparing/ready, live progress board with delay flags
Payments	Cash/card/wallet payment capture, change, daily report, shifts
Menu	Categories and items with availability
Inventory	Stock, low-stock alerts, suppliers, purchase orders, adjustments, item requests
Admin	Staff, restaurants, branches, tables
Analytics	Manager/owner one-page dashboard, exportable reports (Excel/PDF), cashier-scoped reports
Notifications	Role/user-targeted messages with read tracking

4. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	

		Standard pages should render within 2 s on a typical broadband connection; list endpoints return within 500 ms for up to 10,000 orders.
NFR-02	Freshness	Operational views (orders, kitchen, boards, dashboards) auto-refresh every 10 seconds without flicker.
NFR-03	Security	JWT-based auth; bcrypt password hashing; role guards on every protected endpoint; secrets provided via environment variables, never committed.
NFR-04	Usability	Plain-language UI, color-coded statuses, single-screen layouts that minimize scrolling; usable on tablets (≥ 768 px).
NFR-05	Reliability	Backend failures surface explicit errors; the frontend degrades gracefully (empty states, retry on next poll).
NFR-06	Maintainability	TypeScript end-to-end; modular NestJS feature modules; monorepo with shared conventions; `tsc --noEmit` must pass.
NFR-07	Portability	Runs anywhere Node 18+ and PostgreSQL are available; deployable to Replit, Vercel (frontend) + managed Node host (backend), or a single VM.
NFR-08	Auditability	Stock adjustments and item requests record acting users and timestamps.

5. External Interface Requirements

5.1 User Interface

Single-page-app feel with a persistent left sidebar (role-filtered), light gray theme with brand-orange accents (#E8832A), and modal dialogs for POS, payment, order detail, and confirmations.

5.2 API Interface

REST over HTTPS with JSON bodies. The browser-facing path is `/backend/api`; the backend's direct prefix is `/api`. Interactive Swagger documentation exists at backend path `/api/docs` and must be restricted or disabled in production. Main resource groups are `/auth, /users, /restaurants, /branches, /menus, /tables, /orders, /kitchen, /payments, /inventory, /notifications, /summary, /seed`.

5.3 Database Interface

PostgreSQL via TypeORM repositories; connection string in `DATABASE_URL`; SSL honored when `sslmode=require` is present.

6. Assumptions & Dependencies

- A single restaurant company operates the system; multiple branches are supported.
- Kitchen staff record stock deductions/waste for profitability estimates to be accurate.
- Exports (Excel/PDF) are generated client-side (xlsx, jsPDF) and require a modern browser.

7. Acceptance and Production Evidence

Architecture assessment is complete, but repository evidence for the full safe-refactoring, separation, automated-regression, security-hardening, migration, backup/restore, and production-verification stages is incomplete. Requirements marked in this SRS are acceptance targets, not automatic claims of production readiness. Refer to document 12 for current READY / NEEDS ATTENTION / BLOCKED status.